

CIPHERMAIL: SECURE EMAIL SYSTEM WITH AUTONOMOUS SECOPS

Hariprasath R S
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701103@rajalakshmi.edu.in

Hemnath V
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701115@rajalakshmi.edu.in

Mrs M.Priyachitra(SS)
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
priyachitra.m@rajalakshmi.edu.in

Abstract— Email communication is the preferred channel for sharing sensitive information, but traditional email systems offer no end-to-end security against all privacy and security threats. In order to mitigate such problems, this paper presents CipherMail which is a Chrome extension for Gmail that combines privacy-preserving email encryption with email threat detection. The privacy-preserving layer relies on OpenPGP.js library to implement PGP-based encryption, decryption, digital signing, and signature verification operations with all computations taking place in the user's browser. The threat detection layer is implemented as a CipherMail shield in Gmail's email hover toolbar that analyzes emails for suspicious links, sender spoofing, and potentially dangerous attachments. Local heuristic rules are applied together with the VirusTotal API v3 link-reputation analysis to classify threats as Safe, Suspicious, or Dangerous. The proposed system is based on a no-backend approach where keys, settings, and cached threat results are stored locally in the user's browser using Chrome Storage API. The Chrome extension warns users before opening suspicious links and downloading dangerous attachments. A set of automated tests are conducted to verify the correctness of the cryptographic layer. The paper demonstrates how email privacy and threat detection can be built right into the Gmail interface without a need for a separate backend server.

Index Terms— CipherMail, Email Security, PGP Encryption, OpenPGP, Gmail Extension, Phishing Detection, Sender Spoofing, VirusTotal, Threat Detection, Chrome Extension, Privacy Protection

I. INTRODUCTION

Email has become a vital communication channel for exchanging personal, academic, business, and

other sensitive information. While contemporary email services employ secure HTTPS communication and stored messages encryption, an email service provider may still theoretically be able to access the message content. Thus, for users who want to share information that should only be visible to the sender and recipient, this may be a privacy concern. End-to-end encryption aims to address this issue by ensuring the message is encrypted and can only be decrypted by the recipient. Public-key cryptography standards like Pretty Good Privacy (PGP) can be employed to achieve end-to-end encryption. PGP implementations utilize public and private keys to encrypt and decrypt messages, respectively. Additionally, PGP allows creating digital message signatures with a private key for the purpose of verifying the message's origin and integrity. However, using PGP to send encrypted messages typically requires additional software and key-exchange procedures, thus posing a usability challenge. Adding encryption functionalities to the email interface reduces overall complexity and allows encrypting messages while remaining within the familiar environment of an email service.

Besides the privacy concerns, emails pose other security challenges including phishing, malicious links, sender spoofing, and dangerous attachments. A phishing email can contain links and attachments that execute unwanted actions on a user's computer or network. Malicious links and redirections may also appear benign, for instance, if the email's From field or the links themselves contain misleading information. Similarly, attachments that appear harmless can also execute malicious code if their file extension is disguised using double extensions, e.g. exe.csv or similar tricks.

To mitigate these issues, email clients could offer additional information about potentially malicious links and attachments. However, providing such information poses its own set of problems, including

the difficulty of detecting phishing emails, analyzing every suspicious link and attachment in real-time, and ensuring that the process remains transparent and does not disrupt the user. This work describes the development of CipherMail – a Chrome extension for Gmail that implements two features: (i) email encryption/decryption using PGP, and (ii) detecting suspicious links, sender spoofing, and dangerous attachments in emails. The encryption part was implemented using the OpenPGP.js library that provides an API for key generation, encrypting and decrypting messages, and generating and verifying digital signatures. All operations that require user input or that involve sensitive information are performed in the browser, while the Chrome local storage is used to store PGP keys, contacts, and encryption-related settings. No separate backend for CipherMail is needed.

The second part of the proposed solution, which focuses on phishing and malicious content detection, was implemented as a Gmail add-on. A CipherMail shield appears next to email actions (Archive, Mark as read, Delete) in the hover toolbar. When the shield is triggered, the link and attachment analysis features are used to determine if the email is Safe, Suspicious, or Dangerous. Suspicious links are detected using local heuristics that check for the presence of an IP address in the URL, lookalike domains, shortened URLs, and HTTPS; check the From email against the claimed sender's address (if available); and analyze URL against potentially malicious domains using VirusTotal. The attachment name is scanned for suspicious extensions using local heuristic rules. None of the email's content, including body, attachments, or sender information, are sent to CipherMail's VirusTotal API.

The described CipherMail prototype implements the no-backend approach that relies on local encryption and phishing analysis with remote VirusTotal scanning of links for potentially malicious domains. PGP encryption/decryption, local heuristic analysis of links and attachments, and VirusTotal API usage are all implemented as Chrome extension add-ons for Gmail. The proposed solution offers email privacy and security features

within one email extension while eliminating the need to run a separate backend.

II. LITERATURE SURVEY

Email security has been studied from two perspectives: keeping the confidentiality of email content and detecting malicious or deceptive messages. Public-key cryptography has been traditionally used to provide confidentiality, integrity, authentication, and secure key management for electronic communication. OpenPGP defines mechanisms for encryption, decryption, digital signatures, and key management and supports a hybrid approach in which a randomly-generated session key is used to encrypt the message while public-key cryptography protects the session key [1]. These mechanisms provide a foundation for secure email systems in which only the intended recipient can decrypt protected message content.

Although PGP provides strong cryptographic protection, usability and key management have historically been a challenge for ordinary users. Whitten and Tygar conducted a usability evaluation of PGP and reported that many participants were unable to successfully complete secure email operations despite a graphical interface [2]. Their study demonstrated that security mechanisms must not only provide cryptographic protection but should also be integrated into interfaces that users can understand and operate correctly. This observation is especially relevant to browser-based email security systems because security functions can be integrated directly into an email interface rather than having users switch between applications.

Research into email security has focused extensively on phishing detection. Phishing emails typically attempt to deceive users into interacting with malicious websites or providing sensitive information. Literature surveys have investigated approaches based on natural language processing, machine learning, URL analysis, and other features for identifying phishing emails and websites [3], [4]. NLP-based research into phishing emails has explored the use of message content and linguistic characteristics to identify phishing emails, while URL-based approaches study properties of links and their associated domains.

URL analysis is particularly relevant to phishing detection because malicious links are frequently used as an entry point for phishing attacks. Recent studies have examined URL and lexical features, domain information, page content, behavioral characteristics, and hybrid features for identifying phishing websites [4], [5]. These studies

demonstrate that URL-based features can provide useful information for real-time phishing detection while also highlighting challenges caused by evolving

phishing techniques, changing datasets, and the need for lightweight detection methods. More recent research has explored machine-learning and deep-learning approaches for improving the adaptability of phishing detection systems [6].

Email spoofing represents another important security problem in which attackers manipulate sender information to make a message originate from a trusted person or organization. Research into spear-phishing detection has examined sender spoofing, social engineering, malicious links, and malware as interconnected attack techniques. Existing approaches have included signature-based, heuristic, anomaly-based, machine-learning, and hybrid detection mechanisms [7]. These studies demonstrate the importance of examining multiple characteristics of an email rather than relying on a single indicator when determining whether a message may be malicious.

Threat-intelligence and reputation services provide another approach for identifying potentially malicious URLs. Rather than relying entirely on locally-maintained blocklists or manually-defined rules, reputation services can compare URLs against information collected from multiple security engines. Such services can, therefore, complement local analysis by providing external reputation information for links that users encounter. This approach is helpful for browser-based email security because URL reputation can be checked when the user requests a security analysis rather than requiring the complete email message to be uploaded to an external service.

However, many existing studies tend to focus on a particular aspect of email security, such as cryptographic protection, phishing classification, URL detection, or machine-learning-based threat identification. PGP-based systems typically focus on confidentiality and authentication, while phishing-detection research typically focuses on identifying malicious or deceptive content. The usability challenges identified in earlier PGP research also demonstrate the importance of integrating security functions into familiar user interfaces rather than having users deal with complex standalone workflows [2].

This motivates the development of CipherMail, which integrates privacy protection and threat detection within the Gmail interface. Rather than developing a new cryptographic algorithm or independent phishing-classification model, the proposed system combines established technologies into a browser-based security layer. OpenPGP.js is used for PGP key generation, encryption, decryption, signing, and verification, while local heuristic rules analyze suspicious links, sender information, and attachment filenames. VirusTotal link-reputation results are then combined with the local analysis to produce a Safe, Suspicious, or Dangerous classification. The implementation follows a no-backend architecture, with cryptographic processing and local data storage being performed within the browser.

A. Research Gap

Based on the reviewed literature, several research gaps can be identified. Existing PGP research typically provides strong mechanisms for email confidentiality and authentication, but integration into common email workflows remains an important consideration [1], [2]. Phishing-detection research provides numerous approaches for identifying malicious URLs and email content, including heuristic, machine-learning, and deep-learning techniques [3]–[6]. However, these areas are typically treated as separate security problems. A unified browser-based system that combines email encryption with directly-accessible threat analysis within the Gmail interface is less emphasized.

There is also a practical gap between sophisticated phishing-detection research and lightweight, user-controlled browser security tools. Many research approaches depend on trained machine-learning models, large datasets, or extensive feature extraction. For a browser extension intended to operate directly within an email interface, low computational overhead, immediate feedback, and minimal data sharing are important design considerations. CipherMail, therefore, uses local heuristic checks and external URL reputation rather than claiming a new AI-based phishing-detection model. The current implementation focuses on integrating these mechanisms into a practical Gmail workflow.

B. Novelty

The contribution of the proposed work is the integration of PGP-based email privacy and email threat detection within a single Chrome extension for Gmail. The system provides encryption, decryption, signing, and verification functions

within Gmail while also adding a threat-analysis shield to the email hover toolbar. Threat analysis combines local checks for suspicious URLs, sender spoofing, and dangerous attachment extensions with VirusTotal link-reputation information. The system also follows a no-backend architecture, keeping PGP identities, contacts, settings, and cached results within the browser.

C. Objectives

The main objective is to develop and demonstrate a browser-based Gmail security extension that combines secure email communication with proactive threat detection. The specific objectives are to: (i) provide PGP-based encryption and decryption for Gmail messages; (ii) support public and private key generation, import, export, signing, and verification; (iii)

provide local storage for identities, contacts, and security settings; (iv) analyze emails for suspicious links, sender spoofing, and dangerous attachment extensions; (v) integrate VirusTotal API-based URL reputation checking; (vi) classify detected threats as Safe, Suspicious, or Dangerous; and (vii) provide warning prompts before users interact with flagged links or attachments. The proposed system is implemented as a Chrome Manifest V3 extension and evaluated through automated testing of the cryptographic layer.

III. PROPOSED SYSTEM

The proposed system, CipherMail, integrates PGP-based email-encryption and email-threat detection into a Chrome browser extension for Gmail. It provides a privacy and security layer inside the Gmail interface, but requires no dedicated CipherMail backend server. The encryption layer is implemented by using OpenPGP.js to generate and manage PGP keys, and to perform encryption, decryption, signing, and verification. Public keys are used to encrypt messages for a given recipient (the intended reader), and the encrypted message can only be decrypted with the appropriate private key. The private key is always stored encrypted, and only temporarily decrypted (in memory) by the background service worker.

The proposed threat-detection layer adds a CipherMail shield option to Gmail's existing email hover toolbar. The shield option appears alongside Archive, Mark as read, and Delete. On clicking the shield option, content script gets the relevant email information (injected into the DOM), including links, sender details, and attachment files names. It performs some heuristics analysis on the extracted

information. Heuristics analysis looks for suspicious characteristics in links (such as embedded IP addresses), suspicious-looking domains, use of URL shortening services, non-HTTPS links, sender or domain mismatch, and dangerous or double file extensions.

The extracted links are additionally checked against the VirusTotal API v3. The local analysis and VirusTotal reputation are used to determine the final risk score, which maps to Safe, Suspicious, or Dangerous classes. Recent VirusTotal results are cached to reduce repeated requests and stay within the free API limits. When a suspicious link or attachment is found, CipherMail warns the user or asks for confirmation before proceeding.

The system uses a no-backend approach, where identities, contacts, settings, user's VirusTotal API key, and cached threat-detection results are persisted

in the browser using

`'chrome.storage.local'`. Gmail content script communicates with the background service worker (which has access to crypto and VirusTotal APIs) through the Chrome runtime messaging mechanism. Threat analysis and encryption processes always happen on the service worker; the Gmail content script only interacts with the Gmail UI and shows the security analysis results to the user. The content of the email (links, sender details, and attachments) are sent to VirusTotal for analysis, but the body of the email is not uploaded anywhere.

The overall goal of the proposed system is to demonstrate an integrated email-security workflow that adds confidential communication and threat awareness on top of Gmail. The implemented prototype demonstrates key management, encrypted email communication, digital signing and verification, threat analysis, VirusTotal link-reputation checking, and warning mechanisms. The system forms the basis for future improvements such as automatic attachment encryption in Gmail's native attachment flow, keyserver support, true PGP/MIME support, more robust Gmail integration, and additional attachment analysis.

The overall architecture of the proposed CipherMail system is shown below.

IV. SYSTEM ARCHITECTURE

The proposed CipherMail system is based on a browser architecture that uses Gmail, PGP encryption, local browser storage, threat analysis, and VirusTotal link-reputation checking. The

architecture is implemented as a Chrome Manifest V3 extension, and does not require a dedicated CipherMail backend server. Gmail provides the interface, while the Chrome content script and background service worker coordinate encryption, email analysis, storage, and communication with the VirusTotal API.

A. Gmail Interface Layer

This layer provides the interface using Gmail. CipherMail integrates its functionality into the Gmail compose window along with email interface. Encryption and signing controls are added to the compose window, while a CipherMail shield option is added to the email hover toolbar alongside actions such as Archive, Mark as read, and Delete.

B. Gmail Content Script Layer

The content script layer is implemented through `content/gmail.js`, which uses Gmail's Document Object Model (DOM) to detect compose windows, handle encryption and decryption interactions, extract links, sender information, and attachment filenames for threat analysis, and display the corresponding security results. Cryptographic analysis is not performed directly in the Gmail content script; requests are sent instead to the background service worker.

C. The Communication and Background Service

The layer background service worker serves as the central processor of the extension. Chrome runtime messaging is used to communicate between the Gmail content script and the background service worker. The service worker handles cryptographic requests, performs encryption-related operations with the PGP module, and sends VirusTotal requests for link-reputation analysis. It is also the only component where private keys are briefly decrypted in memory during a cryptographic operation.

D. PGP Encryption and Key Management Layer

This layer provides the privacy functionality of CipherMail using `OpenPGP.js`. It supports PGP key-pair generation, message encryption, decryption, digital signing, and signature verification. A public key is used to encrypt the intended recipient's message, while the corresponding private key is required for decryption. The key-management component also supports identities, and imported contact public keys.

E. Local Storage Layer

CipherMail uses the Chrome `chrome.storage.local` API instead of a dedicated database. This local

storage layer stores PGP identities, contacts, application settings, the user's VirusTotal API key, and cached VirusTotal results. Private keys are stored as encrypted key blobs, while passphrases are not. The local-storage approach allows the extension to avoid a CipherMail backend server.

F. Threat Detection Layer

The threat-detection layer analyzes emails using local heuristic rules. It examines links, sender information, and attachment filenames to detect common indicators of potentially malicious content. Link analysis includes checks for IP-address URLs, lookalike domains, URL shorteners, and non-HTTPS links. Sender analysis compares the sender's display name with the actual email domain, while attachment analysis identifies risky extensions such as `.exe`, `.scr`, `.js`, and double extensions such as `.pdf.exe`.

G. VirusTotal Integration Layer

This layer provides external link-reputation information via the VirusTotal API v3. After local analysis is done, links are checked against VirusTotal using the user's own API key. The number of security engines reporting a URL as malicious or suspicious contributes to the overall risk score. Results are cached to reduce repeated requests and help operate within the available API quota. Only link URLs are sent to VirusTotal, while email content and attachments are not.

H. Risk Assessment and Warning Layer

This layer combines the results from local heuristic analysis and VirusTotal reputation checking. The accumulated risk score is mapped to three classifications: Safe, Suspicious, or Dangerous. The results are sent to the user through a security results panel. When potentially dangerous links or attachments are detected, CipherMail provides a warning or confirmation step before the user opens the link or downloads the attachment.

I. Security and Testing Layer

The security layer protects sensitive information during extension operation. Private keys remain encrypted when stored, and are decrypted only briefly in memory during required operations. Decrypted email content is inserted using `textContent` rather than `innerHTML`, and the extension does not use `eval()`. A restrictive Content Security Policy and typed error handling are also implemented. The project also includes 50 automated tests around the cryptographic layer.

J. Risk Assessment and Warning Layer

This layer combines the results from local heuristic analysis and VirusTotal reputation checking. The accumulated risk score is mapped to three classifications: Safe, Suspicious, or Dangerous. The results are sent to the user through a security results panel. When potentially dangerous links or attachments are detected, CipherMail provides a warning or confirmation step before the user opens the link or downloads the attachment.

k. Security and Testing Layer

The security layer protects sensitive information during extension operation. Private keys remain encrypted when stored, and are decrypted only briefly in memory during required operations. Decrypted email content is inserted using `textContent` rather than `innerHTML`, and the extension does not use `eval()`. A restrictive Content Security Policy and typed error handling are also implemented. The project also includes 50 automated tests around the cryptographic layer.

Overall, the architecture provides an end-to-end browser workflow from Gmail → CipherMail content script → background service worker → PGP encryption or threat analysis → local storage / VirusTotal → results returned to Gmail. The architecture keeps the core privacy and security processing inside the user's browser, and only uses VirusTotal as an external link-reputation service. There is no separate CipherMail database, or backend server.

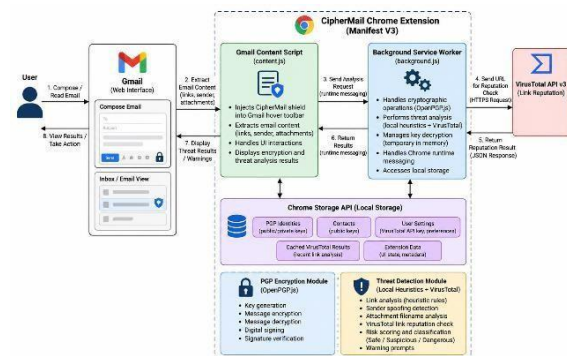


Fig. 1: Overall architecture of the proposed CipherMail system.

V. METHODOLOGY

The proposed system is a browser extension that provides secure email through the Gmail platform, PGP encryption, local analysis of threats, VirusTotal link reputation analysis, and local storage and security warnings. The methodology describes the proposed solution step by step, starting from the interaction in Gmail, through the processing of emails,

encryption or analysis of threats, presenting the results to the user. In general, the processing takes place in the chrome extension, since the proposed solution does not have a dedicated CipherMail server.

a) Gmail Integration

The proposed solution starts in the Gmail interface — CipherMail is a Chrome extension made according to Manifest V3 specifications and adds some features to the Gmail interface. When writing emails, the user is given additional encryption/signing buttons. When reading emails, a CipherMail shield appears in the hover toolbar with Archive, Mark as read, and Delete buttons; thus, the user is not removed from the Gmail interface when working with CipherMail.

b) PGP Key Management and Encryption

The PGP module is implemented with the help of the `openpgp.js` library, which allows generating a pair of private and public keys. The user can save their PGP identity (name, email, passphrase) and import their public keys, as well as the public keys of their contacts. The extension can take the content of the Gmail composition window and encrypt it with the public key of the selected email address. When using `OpenPGP.js`, a hybrid encryption algorithm is used: the message content is encrypted with a randomly selected AES key and the public key of the recipient, which in turn encrypts the AES key.

c) Message Decryption and Verification

While reading the encrypted message, CipherMail detects the encrypted block and decrypts it with the private key and passphrase, which, in turn, are stored locally. If the message was signed with the sender's private key, it is decrypted with the public key. The private key is stored locally in an encrypted form and is only decrypted when the background service worker processes the message. Gmail gets the decrypted message, which is displayed to the user. Similar local operations are performed when verifying signatures and decrypting messages.

d) Email Threat Analysis

When analyzing messages for threats, the user highlights the message with the CipherMail shield; then, the content script extracts some information from the message for analysis. First, this information includes the links in the message, second, the

sender's displayed name and email (domain), and third, the names of the attached files. Then, heuristic checks are applied to this information to check for suspicious content. The following threat detection rules are implemented locally:

- Presence of IP-addresses in the URL of the link;
- Presence of suspicious domains (the same name but different extensions);
- The use of shortening services for URLs;
- The lack of HTTPS for secure connections;
- Mismatch between the displayed email (domain) and the actual one;
- Suspicious extensions of files attached to the message;
- The presence of two extensions in the file name (.pdf.exe).

e) VirusTotal Link Reputation Analysis

After the static checks have been passed, the links in the message are checked against the VirusTotal database using their API. To do this, the cipherMail background service sends a request to VirusTotal's API v3 on behalf of the user with the secret API key. Each link is sent for analysis to determine whether it is malicious or suspicious. The analysis results are displayed to the user in the results panel by assigning a score based on which the link is labeled as safe, suspicious, or dangerous. Since the VirusTotal API allows 2500 requests per month, this is a limiting factor in analyzing each link. To avoid exhausting the quota, the results of the last analysis for each link, as well as the results of heuristic analysis, are stored locally.

f) Risk Scoring and Threat Classification

The results of the heuristic analysis are stored locally; they, along with the results of the VirusTotal analysis, contribute to the calculation of the risk score, based on which the final label (safe, suspicious, dangerous) is selected for the user. Thus, the user gets a simplified view of the analysis: individual items do not affect the score, but several items at once affect the danger level of the message. In addition, the analysis results are visualized in a panel where the user can see which threats were found.

g) Warning, Security Controls, and Testing

Finally, after the analysis of the message is complete, the user receives an alert about the results of the analysis. Suspicious links in the email are highlighted, and a confirmation is required to open

them; suspicious attachments are highlighted, and a confirmation is required to download them. Thus, there are additional security controls in the Gmail interface; however, they do not prevent the default operation of Gmail. The cryptographic part of the extension (encryption/decryption of messages) is subjected to 50 automated tests. Additional security controls include the use of local storage of private keys, processing of content using `textContent`, the absence of `eval()`, the use of Content Security Policy, and the use of typed errors.

VI. RESULTS AND DISCUSSION

The proposed CipherMail system was implemented as a Chrome Manifest V3 extension for Gmail that combined PGP-based email encryption with email threat detection. The implementation demonstrated the full workflow from Gmail interaction to local cryptographic processing, threat analysis, VirusTotal link-reputation checking, and presentation of security results to the user. The current implementation focused on verifying the functionality of the privacy and threat-detection components, rather than reporting large-scale performance measurements. The project documentation confirmed that the cryptographic layer was covered by 50 automated tests. The results can be divided into secure-email output and threat detection output.

a) Secure Email Output

The secure-email functionality was implemented directly into the Gmail compose and email interfaces. Users could generate PGP key pairs, import and export keys, manage contacts' public keys, and use Encrypt/Sign controls in the Gmail compose window. When encrypting, OpenPGP.js used a hybrid encryption process in which the message was encrypted using a randomly generated AES session key and the session key was protected using the recipient's public key. The recipient could subsequently detect the PGP message block and decrypt the message and verify the digital signature when one was present.

The implementation maintained encrypted privatekey material in the local browser storage rather than transmitting it to an external server. Private keys were decrypted only briefly in the background service worker for a required cryptographic operation. This demonstrated the intended privacyfocused design in which the core encryption and decryption operations were within the user's browser

b) Threat Detection Output

The threat-detection functionality was integrated into Gmail's existing hover toolbar through a CipherMail shield option. When the shield was

selected, the extension extracted links, sender information, and attachment filenames from the email. Local heuristic analysis checked for indicators such as IP-address URLs, lookalike domains, URL shorteners, non-HTTPS links, sender-domain mismatches, risky file extensions, and double extensions.

The local analysis was combined with VirusTotal URL reputation results to generate a risk score. The final result was classified as Safe, Suspicious, or Dangerous. Flagged links and attachments triggered warning or confirmation prompts before the user proceeded with the potentially unsafe action. The implementation therefore provided security feedback directly within the Gmail workflow rather than requiring the user to manually inspect suspicious links using a separate service.

c) VirusTotal Integration

The VirusTotal integration was implemented through the VirusTotal API v3. The background service worker sent link URLs for reputation analysis using the user's own API key, while recent results were cached to reduce repeated requests. The system did not upload email bodies, sender information, or attachment files to VirusTotal. This maintained the project's no-backend architecture while providing external reputation information for suspicious links.

d) Overall System Output

The implemented system demonstrated the integration of two security functions within a single Gmail extension: privacy protection through PGP encryption and threat awareness through email analysis. The system successfully provided key management, message encryption and decryption, signing and verification, threat analysis, VirusTotal link checking, risk classification, and warning prompts. The project also maintained a no-backend architecture, with browser-local storage used for identities, contacts, settings, API credentials, and cached results.

The current results demonstrated the functional proof of concept of CipherMail. However, the documentation did not report quantitative measurements such as large-scale detection accuracy, precision, recall, or processing latency. Therefore, such measurements were outside the reported results of the present implementation. Future evaluation could extend the system with more extensive threat datasets, attachment analysis, improved Gmail integration, and security testing. The documented future scope also included keyserver support, true PGP/MIME, more resilient Gmail selectors, and VirusTotal file-hash checking for attachments

VII. CONCLUSION

CipherMail is a browser-based email security extension that combines PGP encryption with threat detection. It supports key generation, email encryption/decryption, digital signatures, contact management, and detection of suspicious links, spoofed senders, and unsafe attachments.

The system uses OpenPGP.js and VirusTotal for security analysis and stores user data locally using Chrome Storage API. It does not require a separate backend server. The current prototype provides warnings for potential threats, while future work includes automatic attachment encryption, keyserver support, improved Gmail integration, and better attachment analysis.

VIII. FUTURE WORK

The Future work can enhance CipherMail by adding automatic attachment encryption, PGP/MIME support, and keyserver integration for easier key management. Threat detection can be improved through VirusTotal file-hash analysis and better identification of phishing links, spoofed senders, and malicious attachments.

Gmail integration can be made more stable by reducing dependency on hardcoded UI selectors, while streaming can improve encryption performance for large files. Further testing with real-world emails, different attachment types, and network conditions can improve reliability. These improvements can strengthen CipherMail while maintaining its no-backend, browser-based design.

XI. REFERENCES

1. A. Whitten and J. D. Tygar, "Why Johnny Can't Encrypt: A Usability Evaluation of PGP 5.0," *Proceedings of the 8th USENIX Security Symposium*, 1999.
2. S. L. Garfinkel and R. C. Miller, "Johnny 2: A User Test of Key Continuity Management with S/MIME and Outlook Express," *Proceedings of the Symposium on Usable Privacy and Security (SOUPS)*, 2005. [verify DOI]
3. S. Fahl, M. Harbach, T. Muders, M. Smith, and U. Sander, "Helping Johnny 2.0 to Encrypt His Facebook Conversations," *Proceedings of the Symposium on Usable Privacy and Security (SOUPS)*, 2012. [verify DOI]
4. S. Ruoti, N. Kim, B. Burgon, T. van der Horst, and K. Seamons, "Confused Johnny: When Automatic Encryption Leads to Confusion and Mistakes," *Proceedings of the Symposium on Usable Privacy and Security (SOUPS)*, 2013. [verify DOI]
5. E. Atwater, C. Bocovich, U. Hengartner, E. Lank, and I. Goldberg, "Leading Johnny to Water: Designing for Usability and Trust," *Proceedings of the Symposium on Usable Privacy and Security (SOUPS)*, 2

6. W. Bai, M. Namara, Y. Qian, P. G. Kelley, M. L. Mazurek, and D. Kim, "An Inconvenient Trust: User Attitudes Toward Security and Usability Tradeoffs for Key-Directory Encryption Systems," *Proceedings of the Symposium on Usable Privacy and Security (SOUPS)*, 2016. [verify DOI]
7. S. Ruoti, J. Andersen, T. Hendershot, D. Zappala, and K. Seamons, "We're on the Same Page: A Usability Study of Secure Email Using Pairs of Novice Users," *Proceedings of the CHI Conference on Human Factors in Computing Systems*, 2016, doi: 10.1145/2858036.2858400. [verify DOI]
8. J. Clark, P. C. van Oorschot, S. Ruoti, K. Seamons, and D. Zappala, "Securing Email," *arXiv preprint*, arXiv:1804.07706, 2018. [verify]
9. J. Callas, L. Donnerhacke, H. Finney, D. Shaw, and R. Thayer, "OpenPGP Message Format," *IETF RFC 4880*, Nov. 2007.
10. P. Huigens, J. Winter, D. Birr-Pixton, and M. Wussler, "OpenPGP," *IETF RFC 9580*, 2024.
11. O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, "Machine Learning Based Phishing Detection From URLs," *Expert Systems with Applications*, vol. 117, 2019. [verify DOI]
12. R. S. Rao and A. R. Pais, "Detection of Phishing Websites Using an Efficient Feature-Based Machine Learning Framework," *Neural Computing and Applications*, vol. 31, 2019. [verify DOI]
13. R. Vinayakumar, K. P. Soman, and P. Poornachandran, "Evaluating Deep Learning Approaches to Characterize and Classify Malicious URLs," *Journal of Intelligent & Fuzzy Systems*, vol. 34, no. 3, 2018. [verify DOI]
14. A. Basit, M. Zafar, X. Liu, A. R. Javed, Z. Jalil, and K. Kifayat, "A Comprehensive Survey of AI-Enabled Phishing Attacks Detection Techniques," *Telecommunication Systems*, vol. 76, 2021, doi: 10.1007/s11235-020-00733-2. [verify DOI]
15. P. Peng, L. Yang, L. Song, and G. Fu, "Opening the Blackbox of VirusTotal: Analyzing Online Phishing Scan Engines," *Proceedings of the ACM Internet Measurement Conference (IMC)*, 2019, doi: 10.1145/3355369.3355585. [verify DOI]
16. Y. Zhang, J. I. Hong, and L. F. Cranor, "Cantina: A Content-Based Approach to Detecting Phishing Web Sites," *Proceedings of the 16th International Conference on World Wide Web (WWW)*, 2007, doi: 10.1145/1242572.1242659. [verify DOI]
17. A. Chhetri et al., "Towards Human-AI Teaming to Mitigate Alert Fatigue in Security Operations Centres," 2024.
18. A. Habibzadeh et al., "Large Language Models for Security Operations Centers: A Comprehensive Survey," 2025. [verify full venue/DOI]
19. N. Carlini, A. P. Felt, and D. Wagner, "An Evaluation of the Google Chrome Extension Security Architecture," *Proceedings of the 21st USENIX Security Symposium*, 2012.
20. A. Barth, A. P. Felt, P. Saxena, and A. Boodman, "Protecting Browsers from Extension Vulnerabilities," *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2010.

IX. RESULTS



Figure 1 CipherMail Setup

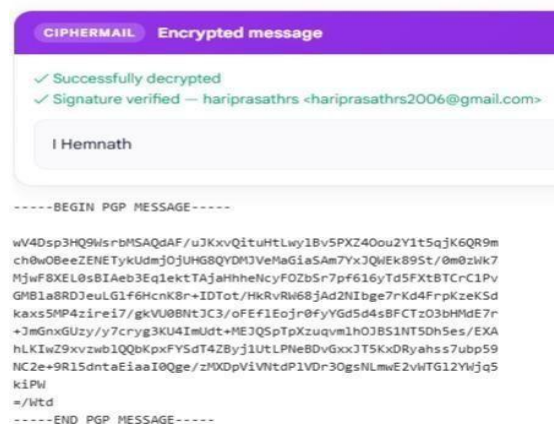


Figure 4 PGP Message Decryption Interface

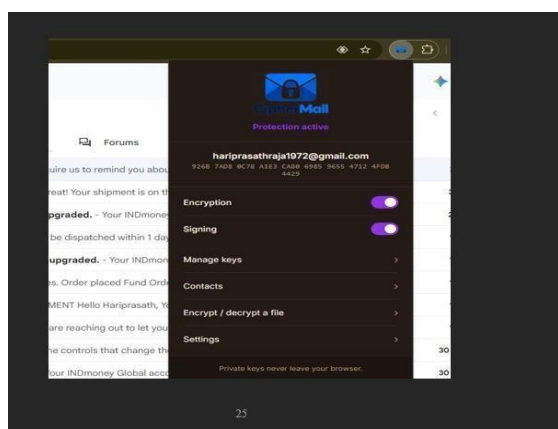


Figure 2 CipherMail Interface

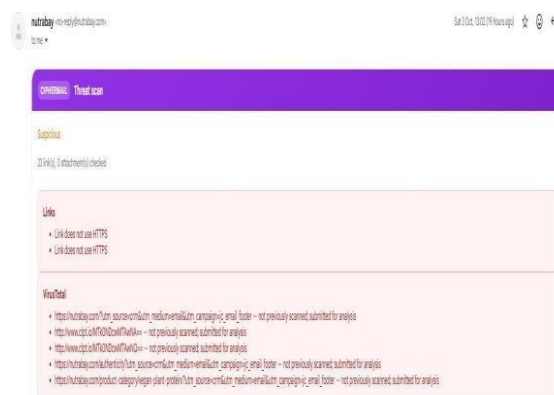


Figure 5 Email Threat Analysis Report

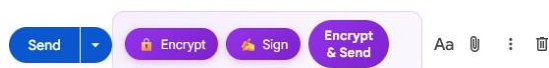


Figure 3 Encrypted Email Composition

